

TerraMind

An AI-Native Development Environment for Terraform Infrastructure

Project concept: A cloud-focused IDE, built as a Code-OSS/VS Code fork or deep extension, that can generate Terraform from natural-language prompts, validate and test the generated infrastructure, predict configuration quality and plan/deployment risk using trained ML models, and use a local coding LLM to explain and repair problems.

1. Problem Statement

Terraform is powerful, but infrastructure code can fail because of invalid references, dependency problems, provider configuration, unsafe permissions, poor architecture, version incompatibilities, and other configuration mistakes. Existing editors mainly assist with syntax and completion. TerraMind proposes an environment specialized for Infrastructure as Code (IaC), where generation, deterministic validation, security analysis, ML-based risk prediction, and AI-assisted remediation are integrated into one workflow.

2. Core User Experience

A user can describe infrastructure in normal language, for example:

```
Create a low-cost AWS backend in ap-south-1 with a VPC, two subnets, an application load balancer, EC2 compute and a private PostgreSQL RDS database. Use secure defaults and make the application publicly reachable over HTTPS.
```

A local coding LLM converts the requirement into structured Terraform files such as **providers.tf**, **variables.tf**, **network.tf**, **compute.tf**, **database.tf** and **outputs.tf**. TerraMind then independently tests and evaluates that output rather than trusting the generator's own assessment.

3. System Pipeline

```
Natural-language prompt -> LLM Terraform generation -> Terraform parser -> terraform fmt -> terraform validate -> terraform plan -> TFLint/Checkov/static rules -> ML feature extraction -> quality/risk prediction -> AI explanation and repair -> re-test.
```

4. Intelligence Layers

Layer	Purpose
Generative AI	Generate Terraform, understand requested changes, explain errors and propose repairs.
Deterministic tooling	Terraform fmt/validate/plan, TFLint and Checkov provide objective checks and machine-readable results.
Machine Learning	Predict plan/deployment risk, configuration quality/risk class, and optionally likely failure category.
Static analysis	Detect known unsafe patterns such as broad IAM permissions, public access, missing encryption and configuration drift.

5. ML Component

Model A - Terraform plan/deployment risk prediction: Binary classification or probability estimation. Input features can include resource count, provider count, module count, dependency depth, cycles, invalid references, variable characteristics, configuration complexity, validation signals, security findings and resource-type distributions. Candidate models: Logistic Regression, Random Forest and XGBoost.

Model B - Failure category prediction: Multiclass classification for categories such as invalid reference, type/configuration error, dependency error, provider/version issue, security/configuration issue and other.

Model C - Infrastructure quality/risk: Predict LOW/MEDIUM/HIGH/CRITICAL risk or a calibrated quality score from code/configuration features. Static scanners should remain the ground-truth source for known deterministic violations; ML should learn broader risk patterns rather than pretending every rule check is an AI prediction.

6. Dataset Strategy

There is unlikely to be one perfect ready-made labeled dataset. The project should create a reproducible dataset pipeline. Collect real public Terraform configurations/modules, preferably restricting the first version to AWS. Parse each configuration and run automated tooling. Generate additional controlled broken variants by introducing missing references, type mismatches, dependency errors, unsafe permissions, provider/version problems and other mutations.

For each sample, record features and labels such as:

```
resource_count | module_count | provider_count | dependency_count | dependency_depth |
dependency_cycles | security_findings | terraform_init_success | terraform_validate_success |
terraform_plan_success | mutation_type | error_type | quality/risk label
```

A reasonable research target could be thousands of real configurations plus multiple controlled mutations per suitable configuration. Actual cloud deployment should only be performed on a small controlled sandbox subset if required; most labeling can use parsing, validation, plan generation and security/static analysis.

7. Local / Free AI Stack

Use **Ollama** to serve a local coding model. Qwen coding models are suitable candidates for Terraform generation, explanation and repair. OpenCode can be used during prototyping or as inspiration for agent/tool orchestration, but TerraMind should own the final workflow. The trained ML models remain separate from the LLM.

```
Suggested stack: Code-OSS or VS Code extension + TypeScript frontend/extension layer +
Python/FastAPI ML service + Ollama/Qwen coder model + Terraform CLI + TFLint + Checkov +
scikit-learn/XGBoost + Pandas.
```

8. Example Output

```
TERRAMIND ANALYSIS Terraform validation: PASS Terraform plan: PASS Static/security checks: 17
/ 20 Predicted quality: 82 / 100 Plan/deployment risk: LOW Failure probability: 8% Potential
issues: - RDS availability could be improved. - IAM policy scope is broader than necessary. -
One security group rule should be restricted. Actions: [Explain] [Generate Fix] [Apply Patch]
[Re-test]
```

9. Recommended Mini-Project Scope

Keep version 1 Terraform-only and preferably AWS-only. The minimum strong implementation should include: (1) prompt-to-Terraform generation, (2) Terraform project parsing and deterministic validation/testing, (3) one genuinely trained ML risk/quality model with proper evaluation metrics, and (4) AI-assisted explanation/repair. Infrastructure graph visualization, cost optimization, multi-cloud support, Kubernetes, Pulumi and CloudFormation should be future-scope features.

Master Build / Research Prompt

The following prompt can be given to a coding/research agent to turn the concept into a concrete architecture and implementation plan.

I want to design and build a college AI/ML + Cloud mini-project called "TerraMind: An AI-Native Development Environment for Terraform Infrastructure."

The product should be a cloud/IaC-focused development environment built either as a Code-OSS/VS Code fork or, if substantially more practical, as a deep VS Code extension with custom panels, diagnostics, commands and an integrated backend. The first version must focus ONLY on Terraform and preferably AWS so that the scope remains achievable.

CORE PRODUCT IDEA

A developer should be able to describe desired cloud infrastructure in natural language. Example: "Create a low-cost AWS backend in ap-south-1 with a VPC, two subnets, an ALB, EC2 compute and a private PostgreSQL RDS database, using secure defaults." A local coding LLM should generate a structured Terraform project (providers.tf, variables.tf, network.tf, compute.tf, database.tf, outputs.tf, etc.).

The generated code must NOT simply be trusted because an LLM created it. TerraMind should independently parse, validate, test and analyze it. The desired pipeline is:

Natural-language prompt

- > local coding LLM
- > generated Terraform
- > Terraform parser/feature extractor
- > terraform fmt
- > terraform validate
- > terraform plan where feasible
- > TFLint
- > Checkov/static security and best-practice rules
- > ML models for quality/risk/failure prediction
- > aggregated analysis
- > LLM explanation and suggested repair
- > patch
- > re-run tests until pass or iteration limit.

GENERATIVE AI

Use a free/local stack where practical. Ollama should be considered as the model runtime and Qwen coding models should be evaluated for Terraform generation, error explanation and code repair. OpenCode may be used for prototyping/inspiration for agentic tool orchestration, but the final project should own the workflow. The LLM is NOT the core trained ML contribution.

MACHINE LEARNING REQUIREMENT

The project must contain genuine ML models trained and evaluated by us, not merely an LLM/API wrapper.

Model A: Terraform plan/deployment risk prediction.

Predict success/failure or a calibrated probability of failure/risk from Terraform/configuration features.

Possible features:

- resource count
- resource type distribution
- module count
- provider count
- variables and outputs
- dependency count
- dependency depth
- dependency cycles
- invalid/missing references
- configuration complexity

- IAM/network/storage characteristics
- Terraform/provider versions
- validation/static-analysis signals
- security violations
- plan-related metadata

Compare appropriate models such as Logistic Regression, Random Forest and XGBoost. Evaluate with accuracy, precision, recall, F1, ROC-AUC/calibration where appropriate.

Model B (if feasible): failure category classification.

Potential classes: invalid reference, type/configuration error, dependency error, provider/version error, IAM/security/configuration issue, other.

Model C (optional): infrastructure quality/risk prediction.

Predict a quality score or LOW/MEDIUM/HIGH/CRITICAL risk class. Do not fake ML where deterministic rules are better. Known issues such as public S3 access, wildcard IAM permissions, open SSH, missing encryption, etc. should be found by static rules/Checkov and can contribute to labels/features. The ML model should learn broader patterns.

DATASET

Assume there is no perfect ready-made dataset. Design a reproducible dataset-generation pipeline.

1. Collect real public Terraform projects/modules, initially AWS-focused.
2. Parse them and extract structured features.
3. Run terraform fmt/validate and, where feasible and safe, terraform init/plan.
4. Run Checkov and TFLint and save machine-readable results.
5. Create controlled mutated/broken versions of suitable Terraform configurations.
6. Mutations should include invalid references, missing variables, wrong types, circular dependencies, bad module references, provider/version incompatibilities, unsafe IAM/network rules, missing encryption, public access, deprecated/invalid attributes and other realistic IaC defects.
7. Re-run the analysis pipeline on each mutation and store the result.
8. Store code/configuration metadata, extracted features, validation/plan result, error type, mutation type, static-analysis findings and labels.
9. Avoid mass real-world terraform apply operations. Actual deployment should only happen in a tightly controlled sandbox subset if needed.

We want enough samples to train a credible model, potentially thousands of original configurations and multiple mutations per suitable sample. Explain how to avoid data leakage: mutated versions of the same base repository/configuration must not be split across train and test sets. Split/group by repository or base configuration.

IDE EXPERIENCE

The editor should show something like:

Terraform validation: PASS

Terraform plan: PASS

Security checks: 17/20

Predicted quality: 82/100

Plan/deployment risk: LOW

Failure probability: 8%

It should highlight affected Terraform lines/resources and allow actions such as:

- Explain issue
- Generate fix
- Apply patch
- Re-test
- Show dependency/infrastructure graph

Natural-language modification should also work. Example: "Make the database highly available" or "reduce cost while keeping availability reasonable." The LLM should produce a patch rather than blindly regenerate the whole project,

followed by the complete validation/testing pipeline.

ARCHITECTURE

Evaluate an architecture around:

- Code-OSS or VS Code extension
- TypeScript for IDE integration
- Python/FastAPI backend for ML/data processing
- Ollama + Qwen coding model
- Terraform CLI
- TFLint
- Checkov
- scikit-learn / XGBoost
- Pandas
- optional graph visualization

The ML predictor and deterministic analyzers must be separate from the LLM generator/evaluator.

WHAT I WANT FROM YOU

Produce a technically realistic implementation blueprint. Include:

1. refined problem statement
2. objectives
3. exact MVP scope
4. features divided into MVP vs future scope
5. system architecture
6. component responsibilities
7. data flow
8. dataset acquisition and generation strategy
9. proposed dataset schema
10. mutation-generation methodology
11. ML feature-engineering plan
12. model selection and evaluation methodology
13. leakage prevention
14. LLM/Ollama integration design
15. Terraform validation/testing pipeline
16. security/static-analysis integration
17. IDE integration strategy
18. recommended repository structure
19. API contracts between IDE/backend/ML/LLM components
20. milestone-by-milestone implementation plan
21. testing strategy
22. major technical risks and mitigations
23. what claims we should NOT make academically
24. demo flow for final presentation
25. realistic division of work for 2-3 students.

Be critical. Do not make the architecture unnecessarily complex just to sound advanced. Clearly identify which features are deterministic software engineering, which are traditional ML, and which are generative AI. The project should remain buildable as a college mini-project while still being technically impressive and academically defensible.